

fid-inc-smoke___fidelity-chain-f-k4-inc

An AgentMPI run analysis

Abstract

This is the deep cell of a smoke test: 47 events, 0.197 s, a `function` executor on all eight ranks, run ten minutes before the broker campaign it was clearing the way for. Its purpose was to confirm that the new `-incompressible` payload generator and the new value-level metrics survive a seven-deep fold, and it does — `fold_depth: 7 = p - 1`, 7 rounds, 7 messages, model check passing, and `n_intact: 96`, `n_label_only: 0`, `n_dropped: 0`, `value_retention: 1.0`. The retention figure of 1.000 is not a fidelity result, and this cell is the one that shows most clearly why. The surrogate kernel copies both of its inputs verbatim into its output and never sees the token budget, so along the chain the outputs run 386, 721, 1,231, 2,092, 3,655, 6,602 and 12,375 tokens against a nominal ceiling of 450, and the fabric moves a 6,617-token accumulator on the final edge — 27,062 output tokens in total for a fold over 96 twelve-token facts. Where a real operator under an enforced budget must eventually discard something, this one simply grows, and `-incompressible` therefore has no mechanism to act on: making a payload unshortenable only matters to an operator that is being squeezed. The single item the kernel did discard, at the first merge, was not a fact at all but the prompt’s own instruction sentence, “such as [F-3-2]. Keep every identifier ...”, which the blob for the $6 \rightarrow 5$ message shows absent while both real reports are intact. What this run establishes, then, is that a surrogate executor cannot exercise whole-rank erasure, however deep the fold. The erasure measurement is `inc-mb-fidelity-inc__fidelity-chain-f-k4-inc`: same algorithm, same p , same nominal budget, but enforced and on real agents, and there retention is 0.3854 with ranks 4 through 7 gone.

1 What this run is

property	value
run	fid-inc-smoke__fidelity-chain-f-k4-inc
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	<code>function</code> $\times$ 8
events	47
wall time	0.20 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.05×	total busy time over wall time
parallel efficiency	0.6%	achieved parallelism per rank
idle fraction	99.4%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	3.56×	slowest rank’s busy time over the mean
coordination share	37.4%	rank-seconds blocked in collectives, of those available
coordination in flight	97.7%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	14,950	0 deferred under rendezvous
tokens in / out	16,844 / 27,062	prompt and completion
cost	\$0.4565	0.0169 \$/kTok out

3 What the analysis notices

- [\[note\]](#) Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- [\[note\]](#) Load imbalance is 3.56×: the slowest rank was busy far longer than the mean.
- [\[ok\]](#) All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- [\[ok\]](#) All 1 checkable collective(s) also matched the predicted round count.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	2.1	1	1	0	1	0	0.0	finalized
1	0.00	1.4	1	1	1	1	6,617	0.0	finalized
2	0.00	1.0	1	1	1	1	3,670	0.0	finalized
3	0.00	1.4	1	1	1	1	2,107	0.0	finalized
4	0.00	2.0	1	1	1	1	1,246	0.0	finalized
5	0.00	0.0	0	1	1	1	736	0.0	finalized
6	0.00	0.0	0	1	1	1	401	0.0	finalized
7	0.00	0.0	0	0	1	0	173	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

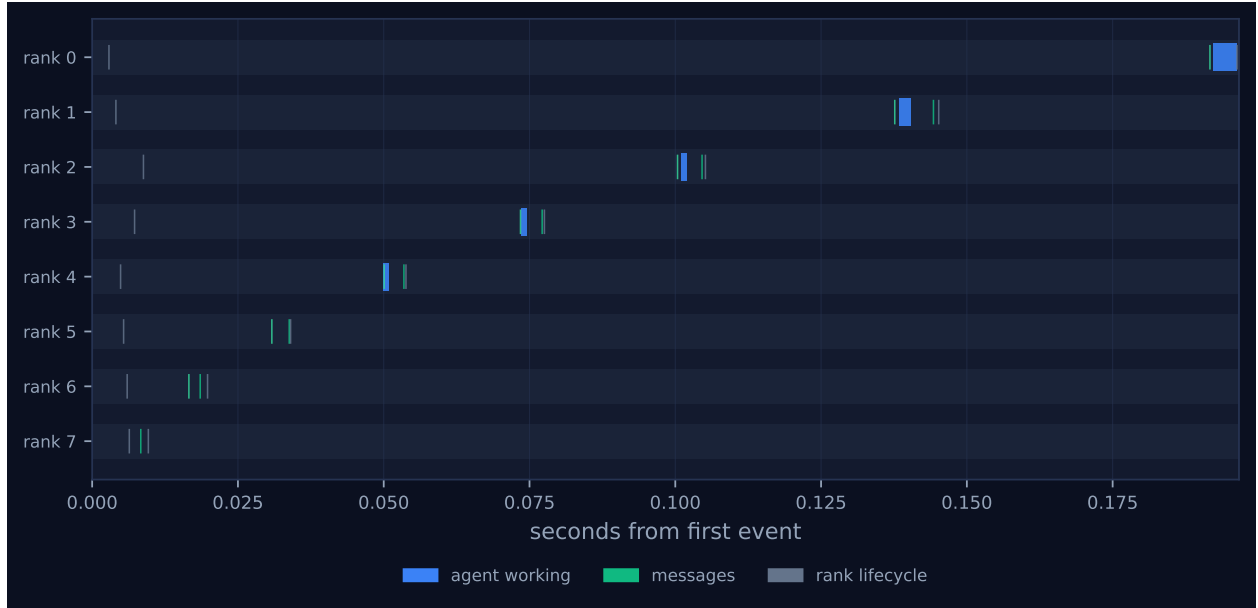


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer’s palette so the same shapes read the same way in both.

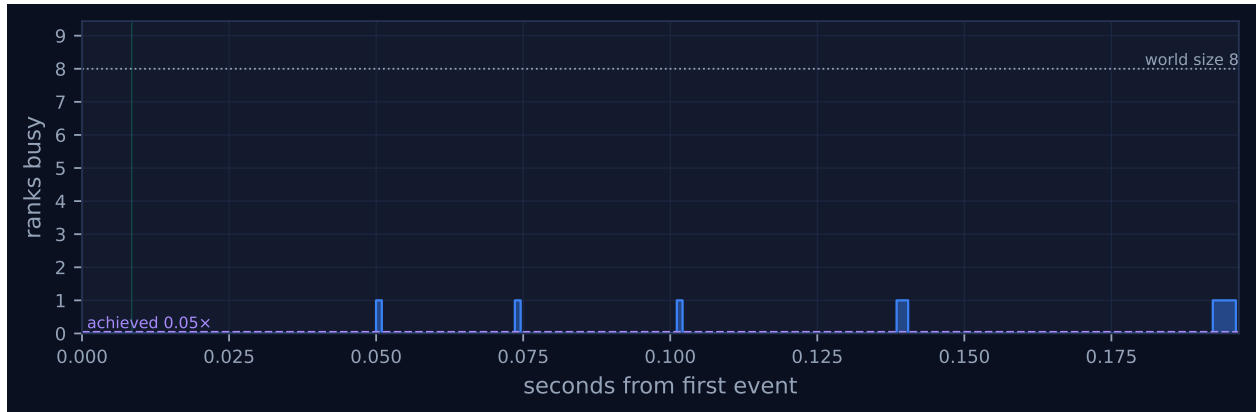


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

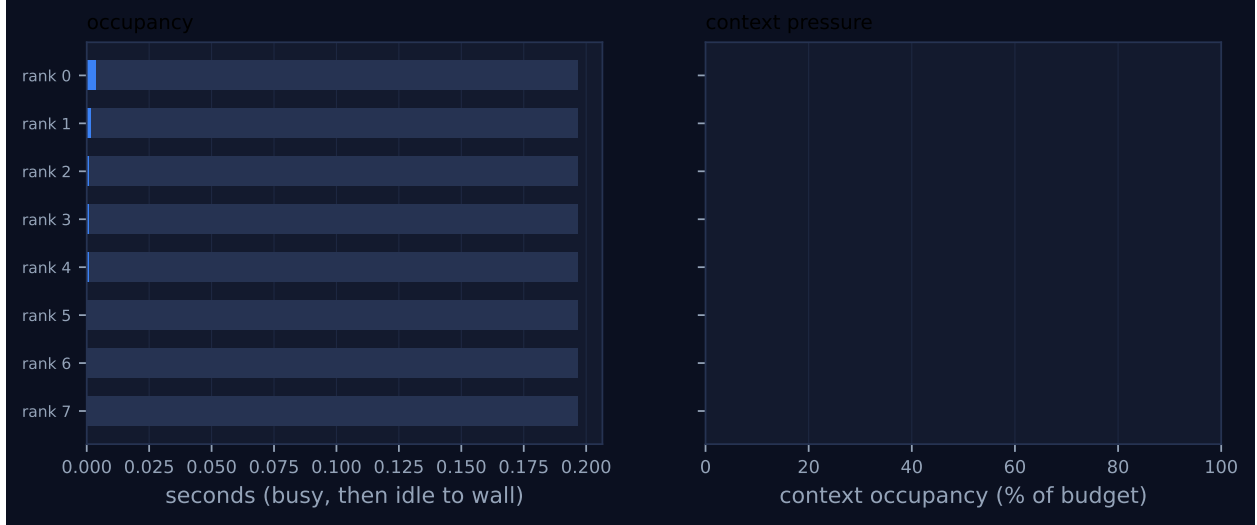


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	chain	—	8	8	7	7	7	7	14,845	0.19	0.19	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is a short diagonal: rank 7 acts at $t = 0.008$ s, rank 6 at 0.008, then 5, 4, 3, 2 and finally rank 0 at 0.193 s, one bar per lane and never two at once. That is the chain reduction with nothing else in it, and the peak-ranks-busy figure of 1 of 8 with a 100% serial fraction is the algorithm’s definition rather than a defect. The $3.56\times$ imbalance flagged in the findings is the mildest in the campaign only because the surrogate’s per-call cost grows with the accumulator, so the deeper ranks accumulate more of the (microsecond-scale) busy time; the occupancy figures themselves should be ignored, since a **function** executor emits no broker claim events and the generated findings say so.

The widths accelerate sharply and that is the run’s most visible property. Per-call latencies are 0.000 s for the first four merges and then 0.001, 0.002 and 0.004 s, so the root’s application alone is a fifth of the run, on a 6,887-token prompt returning 12,375 tokens. The communication matrix, Figure 4, is the matching diagonal band with weights growing 173, 401, 736, 1,246, 2,107, 3,670, 6,617 tokens up the chain, 14,950 in total against the binomial sibling’s 2,579 and the k -ary sibling’s 1,868. The chain is the arrangement in which a copy-through operator hurts most, because every application re-ingests everything that came before.

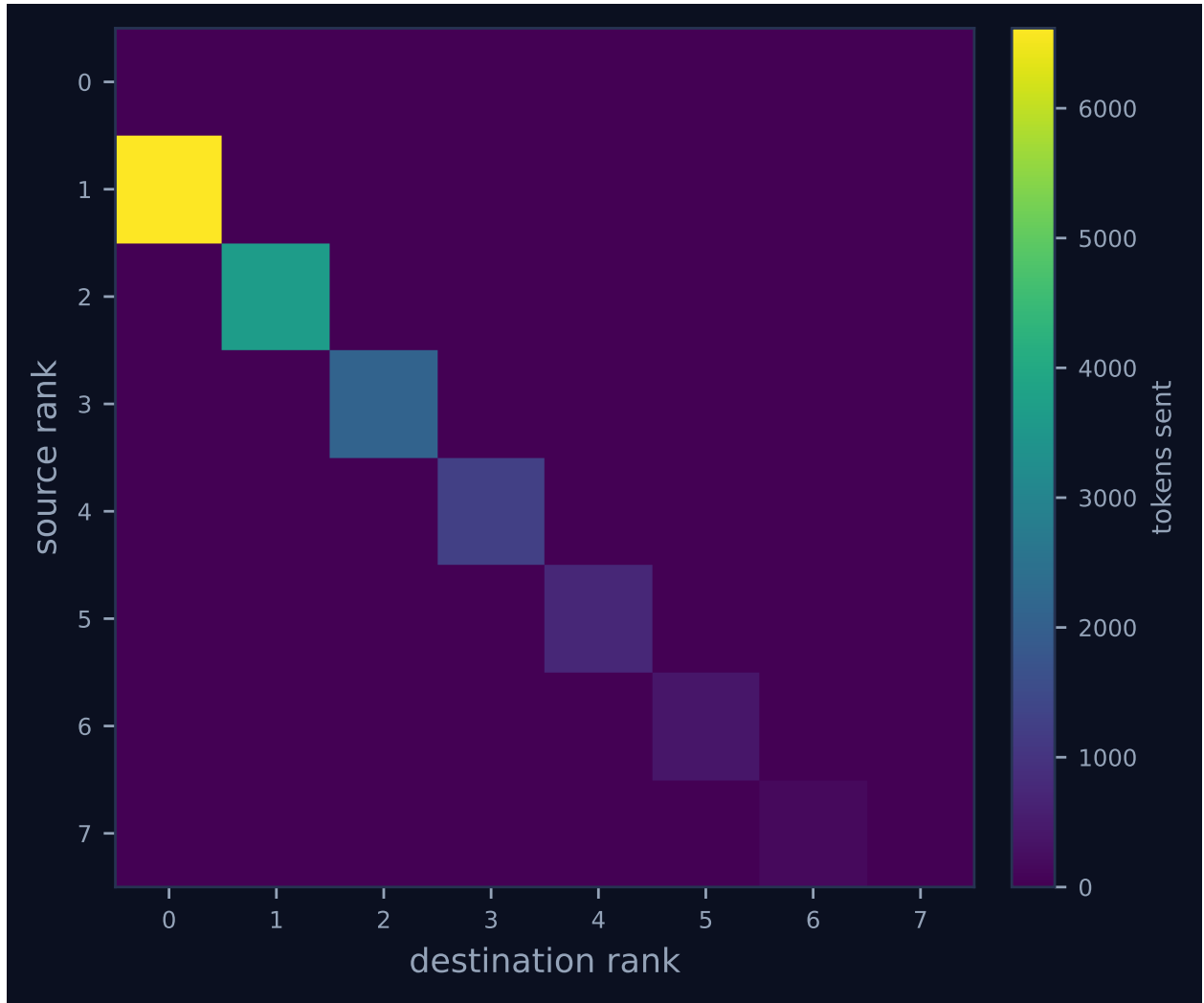


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

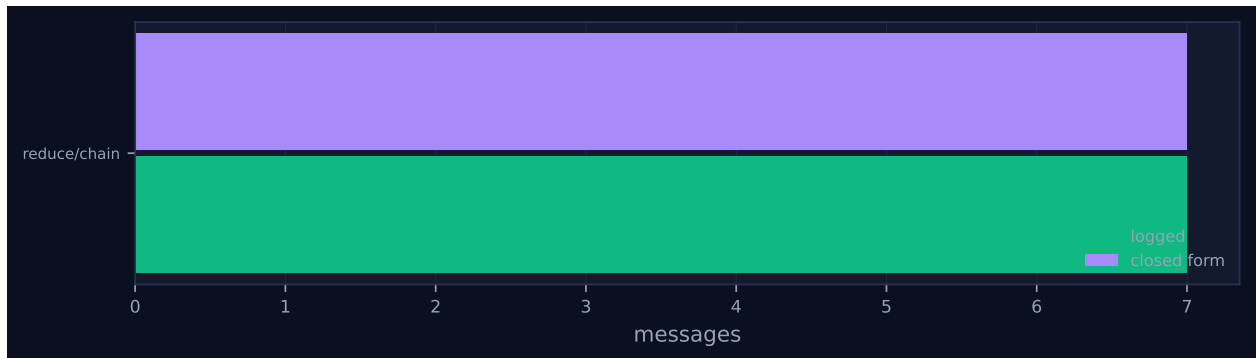


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

7 Interpretation

The structure is by construction. `fold_depth: 7` equals $p - 1$, matching the `chain` branch of `algorithms.py`, and this is the deep end of the smoke test’s depth range, which spans 2 (`kary`), 3 (`binomial`) and 7 — three algorithms rather than four, since `flat` was omitted from `-algorithms` for this run. Seven messages against a predicted seven and seven rounds against a predicted seven are the model check passing. As a smoke test that is the whole of the positive result: the incompressible generator produces twelve distinct high-entropy items per rank, the identifiers survive the plumbing, and the `value_retention` block computes.

What the run establishes beyond that is a limitation of the instrument. Whole-rank erasure in this experiment is what happens when an accumulator standing for many ranks fills a binding output budget: the later contributions are not compressed but dropped entire. Two things must be true for that to happen, and neither is true here. The budget is not binding — at commit `90ca469` the harness passed the module-level `MERGE_CONTRACT`, which has no `max_tokens` field, so the 450 tokens were a sentence in the prompt and a `meta` hint, which is why a 12,375-token output records `attempt: 1` and the run reports zero contract violations. And the operator is not budget-aware: `_surrogate_merge` receives a drop rate and a seed and nothing else, so it grows without limit rather than choosing what to shed. Against that background, `-incompressible` is inert. Its point is to close the escape route the broker runs took — `sat-mb-fidelity`’s real agents re-encoded 96 items from 123.6 bytes per identifier down to 14.6 and then overran the budget by 54%, so nothing was ever forced out — and closing a route the surrogate never used changes nothing.

The kernel’s actual loss behaviour is also worth recording, because it is not what its docstring claims and this cell is a clean instance. It builds candidates from `prompt.splitlines()` and keeps lines matching `FACT_ID`, but each input is interpolated as `json.dumps(...)` and contains no newlines, so a binary merge prompt offers exactly three candidates: the instruction line carrying the example identifier `[F-3-2]`, and the two inputs entire. The unit of loss is a whole input, not a fact, so the docstring’s independent per-item drop and its $(1 - r)^{\text{depth}}$ curve do not describe what runs. Here the seven applications produced exactly one discard, at depth 1, and the blob for the $6 \rightarrow 5$ message identifies it: two findings, 24 identifiers, ranks 6 and 7 complete, phantom absent. Every subsequent payload carries the full prefix — 24, 37, 49, 60, 72, 84 identifiers, where the odd extra one at depths 2 and 3 is the phantom `[F-3-2]`, which survived the draw at rank 5’s merge and is then absorbed into rank 3’s own set when rank 3 contributes at depth 4 — so nothing real was lost in seven applications. Compare the $p = 16$ surrogate chain in `fid-synth`, where a single unlucky draw at depth 4 took the whole accumulated tail of ranks 12 to 15 in one application, sixteen items, and the output collapsed from 862 tokens to 180. That is the shape of this loss model when it does fire: not a gradient, a cliff.

8 Threats to this reading

The executor is a surrogate, so nothing here is agent behaviour. The \$0.4565 is a modelled price for tokens produced by concatenation, and the 27,062 output tokens are the strongest reason not to pool this run’s cost with any broker run’s: the real chain at $p = 8$ on the same twelve items per rank emitted 2,623 output tokens.

The reconstruction is indirect. A `function` executor writes no rows to `agent_calls`, so the fabric holds no prompts and no results; I read the seven message payloads under `blobs/` and inferred each application’s effect by differencing consecutive identifier sets along the chain. That is sound for the

interior but says nothing about the root’s own final application, whose result was never sent — the claim that rank 0 survived rests on the results file’s `per_rank_retention`, not on a blob. I also did not replay the kernel’s draws for this cell as I did for the binomial sibling, where four prompts of identical length predict the observed discard exactly; here the seven prompt lengths are all distinct (446 to 6,887 tokens), so the single discard is consistent with an independent draw at $r = 0.1$ over seven applications and I have no reason to attribute it to the length-seeding defect.

Finally, `value_retention: 1.0` and `n_label_only: 0` are the metrics whose first exercise this run is, and a smoke test that returns the trivial answer does not demonstrate they can return a non-trivial one. That demonstration is `inc-mb-fidelity-inc`, where `n_dropped: 59` and `n_label_only: 0` together establish that the enforced-budget failure is clean deletion rather than label-without-payload survival — which is the distinction the new metric was added to make.